JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class ForkJoinTask<V>

java.lang.Object
    java.util.concurrent.ForkJoinTask<V>

Type Parameters:

V - the type of the result of the task

All Implemented Interfaces:

Serializable, Future<V>

Direct Known Subclasses:

CountedCompleter, RecursiveAction, RecursiveTask

```
public abstract class ForkJoinTask<V>
extends Object
implements Future<V>, Serializable
```

Abstract base class for tasks that run within a ForkJoinPool. A ForkJoinTask is a thread-like entity that is much lighter weight than a normal thread. Huge numbers of tasks and subtasks may be hosted by a small number of actual threads in a ForkJoinPool, at the price of some usage limitations.

A "main" ForkJoinTask begins execution when it is explicitly submitted to a ForkJoinPool, or, if not already engaged in a ForkJoin computation, commenced in the ForkJoinPool.commonPool() via fork(), invoke(), or related methods. Once started, it will usually in turn start other subtasks. As indicated by the name of this class, many programs using ForkJoinTask employ only methods fork() and join(), or derivatives such as invokeAll. However, this class also provides a number of other methods that can come into play in advanced usages, as well as extension mechanics that allow support of new forms of fork/join processing.

A ForkJoinTask is a lightweight form of Future. The efficiency of ForkJoinTasks stems from a set of restrictions (that are only partially statically enforceable) reflecting their main use as computational tasks calculating pure functions or operating on purely isolated objects. The primary coordination mechanisms are fork(), that arranges asynchronous execution, and join(), that doesn't proceed until the task's result has been computed. Computations should ideally avoid synchronized methods or blocks, and should minimize other blocking synchronization apart from joining other tasks or using synchronizers such as Phasers that are advertised to cooperate with fork/join scheduling. Subdividable tasks should also not perform blocking I/O, and should ideally access variables that are completely independent of those accessed by other running tasks. These guidelines are loosely enforced by not permitting checked exceptions such as IOExceptions to be thrown. However, computations may still encounter unchecked exceptions, that are rethrown to

callers attempting to join them. These exceptions may additionally include `RejectedExecutionException` stemming from internal resource exhaustion, such as failure to allocate internal task queues. Rethrown exceptions behave in the same way as regular exceptions, but, when possible, contain stack traces (as displayed for example using `ex.printStackTrace()`) of both the thread that initiated the computation as well as the thread actually encountering the exception; minimally only the latter.

It is possible to define and use ForkJoinTasks that may block, but doing so requires three further considerations: (1) Completion of few if any *other* tasks should be dependent on a task that blocks on external synchronization or I/O. Event-style async tasks that are never joined (for example, those subclassing `CountedCompleter`) often fall into this category. (2) To minimize resource impact, tasks should be small; ideally performing only the (possibly) blocking action. (3) Unless the `ForkJoinPool.ManagedBlocker` API is used, or the number of possibly blocked tasks is known to be less than the pool's `ForkJoinPool.getParallelism()` level, the pool cannot guarantee that enough threads will be available to ensure progress or good performance.

The primary method for awaiting completion and extracting results of a task is `join()`, but there are several variants: The `Future.get()` methods support interruptible and/or timed waits for completion and report results using `Future` conventions. Method `invoke()` is semantically equivalent to `fork(); join()` but always attempts to begin execution in the current thread. The "*quiet*" forms of these methods do not extract results or report exceptions. These may be useful when a set of tasks are being executed, and you need to delay processing of results or exceptions until all complete. Method `invokeAll` (available in multiple versions) performs the most common form of parallel invocation: forking a set of tasks and joining them all.

In the most typical usages, a fork-join pair act like a call (fork) and return (join) from a parallel recursive function. As is the case with other forms of recursive calls, returns (joins) should be performed innermost-first. For example, `a.fork(); b.fork(); b.join(); a.join();` is likely to be substantially more efficient than joining `a` before `b`.

The execution status of tasks may be queried at several levels of detail: `isDone()` is true if a task completed in any way (including the case where a task was cancelled without executing); `isCompletedNormally()` is true if a task completed without cancellation or encountering an exception; `isCancelled()` is true if the task was cancelled (in which case `getException()` returns a `CancellationException`); and `isCompletedAbnormally()` is true if a task was either cancelled or encountered an exception, in which case `getException()` will return either the encountered exception or `CancellationException`.

The ForkJoinTask class is not usually directly subclassed. Instead, you subclass one of the abstract classes that support a particular style of fork/join processing, typically `RecursiveAction` for most computations that do not return results, `RecursiveTask` for those that do, and `CountedCompleter` for those in which completed actions trigger other actions. Normally, a concrete ForkJoinTask subclass declares fields comprising its parameters, established in a constructor, and then defines a `compute` method that somehow uses the control methods supplied by this base class.

Method `join()` and its variants are appropriate for use only when completion dependencies are acyclic; that is, the parallel computation can be described as a directed acyclic graph (DAG). Otherwise, executions may encounter a form of deadlock as tasks cyclically wait for each other. However, this framework supports other methods and techniques (for example the use of `Phaser`, `helpQuiesce()`, and `complete(V)`) that may be of use in constructing custom subclasses for problems that are not statically structured as DAGs. To support such usages, a ForkJoinTask may be atomically *tagged* with a `short` value using `setForkJoinTaskTag(short)` or `compareAndSetForkJoinTaskTag(short, short)` and checked using `getForkJoinTaskTag()`. The ForkJoinTask implementation does not use these `protected` methods or tags for any purpose, but they may be of use in the construction of specialized subclasses. For example, parallel graph traversals can use the supplied methods to avoid revisiting nodes/tasks that have already been processed. (Method names for tagging are bulky in part to encourage definition of methods that reflect their usage patterns.)

Most base support methods are `final`, to prevent overriding of implementations that are intrinsically tied to the underlying lightweight task scheduling framework. Developers creating new basic styles of fork/join processing should minimally implement `protected` methods `exec()`, `setRawResult(V)`, and `getRawResult()`, while also introducing an abstract computational method that can be implemented in its subclasses, possibly relying on other `protected` methods provided by this class.

ForkJoinTasks should perform relatively small amounts of computation. Large tasks should be split into smaller subtasks, usually via recursive decomposition. As a very rough rule of thumb, a task should perform more than 100 and less than 10000 basic computational steps, and should avoid indefinite looping. If tasks are too big, then parallelism cannot improve throughput. If too small, then memory and internal task maintenance overhead may overwhelm processing.

This class provides `adapt` methods for `Runnable` and `Callable`, that may be of use when mixing execution of `ForkJoinTasks` with other kinds of tasks. When all tasks are of this form, consider using a pool constructed in *asyncMode*.

ForkJoinTasks are `Serializable`, which enables them to be used in extensions such as remote execution frameworks. It is sensible to serialize tasks only before or after, but not during, execution. Serialization is not relied on during execution itself.

Since:

1.7

See Also:

Serialized Form

Nested Class Summary

Nested classes/interfaces declared in interface Future
`Future.State`

Constructor Summary

Constructors

Constructor	Description
`ForkJoinTask()`	Constructor for subclasses to call.

Method Summary

All Methods | **Static Methods**

Modifier and Type	Method	Description
`static ForkJoinTask<?>`	**`adapt`**`(Runnable runnable)`	Returns a new `ForkJoinTask` that performs the `run` method of the given `Runnable` as its action, and returns a null result upon `join()`.
`static <T> ForkJoinTask<T>`	**`adapt`**`(Runnable runnable, T result)`	Returns a new `ForkJoinTask` that performs the `run` method of the given `Runnable` as its action, and returns the given result upon `join()`.

static <T> ForkJoinTask<T>	**adapt**(Callable<? extends T> callable)	Returns a new ForkJoinTask that performs the call method of the given Callable as its action, and returns its result upon join(), translating any checked exceptions encountered into RuntimeException.
static ForkJoinTask<?>	**adaptInterruptible**(Runnable runnable)	Returns a new ForkJoinTask that performs the run method of the given Runnable as its action, and returns null upon join(), translating any checked exceptions encountered into RuntimeException.
static <T> ForkJoinTask<T>	**adaptInterruptible**(Runnable runnable, T result)	Returns a new ForkJoinTask that performs the run method of the given Runnable as its action, and returns the given result upon join(), translating any checked exceptions encountered into RuntimeException.
static <T> ForkJoinTask<T>	**adaptInterruptible**(Callable<? extends T> callable)	Returns a new ForkJoinTask that performs the call method of the given Callable as its action, and returns its result upon join(), translating any checked exceptions encountered into RuntimeException.
boolean	**cancel**(boolean mayInterruptIfR	Attempts to cancel execution of this task.

final boolean	**compareAndSetForkJoinTas**(short expect, short update)	Atomically conditionally sets the tag value for this task.
void	**complete**(V value)	Completes this task, and if not already aborted or cancelled, returning the given value as the result of subsequent invocations of `join` and related operations.
void	**completeExceptionally**(Throwable ex)	Completes this task abnormally, and if not already aborted or cancelled, causes it to throw the given exception upon `join` and related operations.
Throwable	**exceptionNow**()	Returns the exception thrown by the task, without waiting.
protected abstract boolean	**exec**()	Immediately performs the base action of this task and returns true if, upon return from this method, this task is guaranteed to have completed.
final ForkJoinTask<V>	**fork**()	Arranges to asynchronously execute this task in the pool the current task is running in, if applicable, or using the ForkJoinPool.commonPool() if not inForkJoinPool().
final V	**get**()	Waits if necessary for the computation to complete, and then retrieves its result.

final V	**get**(long timeout, TimeUnit unit)	Waits if necessary for at most the given time for the computation to complete, and then retrieves its result, if available.
final Throwable	**getException**()	Returns the exception thrown by the base computation, or a `CancellationException` if cancelled, or `null` if none or if the method has not yet completed.
final short	**getForkJoinTaskTag**()	Returns the tag for this task.
static ForkJoinPool	**getPool**()	Returns the pool hosting the current thread, or `null` if the current thread is executing outside of any ForkJoinPool.
static int	**getQueuedTaskCount**()	Returns an estimate of the number of tasks that have been forked by the current worker thread but not yet executed.
abstract V	**getRawResult**()	Returns the result that would be returned by `join()`, even if this task completed abnormally, or `null` if this task is not known to have been completed.
static int	**getSurplusQueuedTaskCoun**	Returns an estimate of how many more locally queued tasks are held by the current worker thread than there are other worker threads that might steal them, or

		zero if this thread is not operating in a ForkJoinPool.
static void	**helpQuiesce**()	Possibly executes tasks until the pool hosting the current task is quiescent.
static boolean	**inForkJoinPool**()	Returns `true` if the current thread is a `ForkJoinWorkerThread` executing as a ForkJoinPool computation.
final V	**invoke**()	Commences performing this task, awaits its completion if necessary, and returns its result, or throws an (unchecked) `RuntimeException` or `Error` if the underlying computation did so.
static <T extends ForkJoinTask<?>> Collection<T>	**invokeAll**(Collection<T> tasks)	Forks all tasks in the specified collection, returning when `isDone` holds for each task or an (unchecked) exception is encountered, in which case the exception is rethrown.
static void	**invokeAll**(ForkJoinTask<?>... tasks)	Forks the given tasks, returning when `isDone` holds for each task or an (unchecked) exception is

		encountered, in which case the exception is rethrown.
static void	**invokeAll**(ForkJoinTask<?> t1, ForkJoinTask<?> t2)	Forks the given tasks, returning when isDone holds for each task or an (unchecked) exception is encountered, in which case the exception is rethrown.
final boolean	**isCancelled**()	Returns true if this task was cancelled before it completed normally.
final boolean	**isCompletedAbnormally**()	Returns true if this task threw an exception or was cancelled.
final boolean	**isCompletedNormally**()	Returns true if this task completed without throwing an exception and was not cancelled.
final boolean	**isDone**()	Returns true if this task completed.
final V	**join**()	Returns the result of the computation when it is done.
protected static ForkJoinTask<?>	**peekNextLocalTask**()	Returns, but does not unschedule or execute, a task queued by the current thread but not yet executed, if one is immediately available.
protected static ForkJoinTask<?>	**pollNextLocalTask**()	Unschedules and returns, without executing, the next task

		queued by the current thread but not yet executed, if the current thread is operating in a ForkJoinPool.
protected static ForkJoinTask<?>	**pollSubmission**()	If the current thread is operating in a ForkJoinPool, unschedules and returns, without executing, a task externally submitted to the pool, if one is available.
protected static ForkJoinTask<?>	**pollTask**()	If the current thread is operating in a ForkJoinPool, unschedules and returns, without executing, the next task queued by the current thread but not yet executed, if one is available, or if not available, a task that was forked by some other thread, if available.
final void	**quietlyComplete**()	Completes this task normally without setting a value.
final void	**quietlyInvoke**()	Commences performing this task and awaits its completion if necessary, without returning its result or throwing its exception.

final void	**quietlyJoin**()	Joins this task, without returning its result or throwing its exception.
final boolean	**quietlyJoin**(long timeout, TimeUnit unit)	Tries to join this task, returning true if it completed (possibly exceptionally) before the given timeout elapsed and the current thread has not been interrupted.
final boolean	**quietlyJoinUninterruptib**(long timeout, TimeUnit unit)	Tries to join this task, returning true if it completed (possibly exceptionally) before the given timeout elapsed.
void	**reinitialize**()	Resets the internal bookkeeping state of this task, allowing a subsequent `fork`.
V	**resultNow**()	Returns the computed result, without waiting.
final short	**setForkJoinTaskTag**(short newValue)	Atomically sets the tag value for this task and returns the old value.
protected abstract void	**setRawResult**(V value)	Forces the given value to be returned as a result.
Future.State	**state**()	Returns the computation state.
boolean	**tryUnfork**()	Tries to unschedule this task for execution.

Methods declared in class Object

`clone`, `equals`, `finalize`, `getClass`, `hashCode`, `notify`, `notifyAll`, `toString`, `wait`, `wait`, `wait`

Constructor Details

ForkJoinTask

```
public ForkJoinTask()
```

Constructor for subclasses to call.

Method Details

fork

```
public final ForkJoinTask<V> fork()
```

Arranges to asynchronously execute this task in the pool the current task is running in, if applicable, or using the `ForkJoinPool.commonPool()` if not `inForkJoinPool()`. While it is not necessarily enforced, it is a usage error to fork a task more than once unless it has completed and been reinitialized. Subsequent modifications to the state of this task or any data it operates on are not necessarily consistently observable by any thread other than the one executing it unless preceded by a call to `join()` or related methods, or a call to `isDone()` returning `true`.

Returns:

`this`, to simplify usage

join

```
public final V join()
```

Returns the result of the computation when it is done. This method differs from `get()` in that abnormal completion results in `RuntimeException` or `Error`, not `ExecutionException`, and that interrupts of the calling thread do *not* cause the method to abruptly return by throwing `InterruptedException`.

Returns:

the computed result

invoke

```
public final V invoke()
```

Commences performing this task, awaits its completion if necessary, and returns its result, or throws an (unchecked) `RuntimeException` or `Error` if the underlying computation did so.

Returns:

the computed result

invokeAll

```
public static void invokeAll(ForkJoinTask<?> t1,
                             ForkJoinTask<?> t2)
```

Forks the given tasks, returning when `isDone` holds for each task or an (unchecked) exception is encountered, in which case the exception is rethrown. If more than one task encounters an exception, then this method throws any one of these exceptions. If any task encounters an exception, the other may be cancelled. However, the execution status of individual tasks is not guaranteed upon exceptional return. The status of each task may be obtained using `getException()` and related methods to check if they have been cancelled, completed normally or exceptionally, or left unprocessed.

Parameters:

`t1` - the first task

`t2` - the second task

Throws:

`NullPointerException` - if any task is null

invokeAll

```
public static void invokeAll(ForkJoinTask<?>... tasks)
```

Forks the given tasks, returning when `isDone` holds for each task or an (unchecked) exception is encountered, in which case the exception is rethrown. If more than one task encounters an exception, then this method throws any one of these exceptions. If any task encounters an exception, others may be cancelled. However, the execution status of individual tasks is not guaranteed upon exceptional return. The status of each task may be obtained using `getException()` and related methods to check if they have been cancelled, completed normally or exceptionally, or left unprocessed.

Parameters:

`tasks` - the tasks

Throws:

`NullPointerException` - if any task is null

invokeAll

```
public static <T extends ForkJoinTask<?>>
Collection<T> invokeAll(Collection<T> tasks)
```

Forks all tasks in the specified collection, returning when `isDone` holds for each task or an (unchecked) exception is encountered, in which case the exception is rethrown. If more than one task encounters an exception, then this method throws any one of these exceptions. If any task encounters an exception, others may be cancelled. However, the execution status of individual tasks is not guaranteed upon exceptional return. The status of each task may be obtained using `getException()` and related methods to check if they have been cancelled, completed normally or exceptionally, or left unprocessed.

Type Parameters:

`T` - the type of the values returned from the tasks

Parameters:

`tasks` - the collection of tasks

Returns:

the tasks argument, to simplify usage

Throws:

`NullPointerException` - if tasks or any element are null

cancel

```
public boolean cancel(boolean mayInterruptIfRunning)
```

Attempts to cancel execution of this task. This attempt will fail if the task has already completed or could not be cancelled for some other reason. If successful, and this task has not started when `cancel` is called, execution of this task is suppressed. After this method returns successfully, unless there is an intervening call to `reinitialize()`, subsequent calls to `isCancelled()`, `isDone()`, and `cancel` will return `true` and calls to `join()` and related methods will result in `CancellationException`.

This method may be overridden in subclasses, but if so, must still ensure that these properties hold. In particular, the `cancel` method itself must not throw exceptions.

This method is designed to be invoked by *other* tasks. To terminate the current task, you can just return or throw an unchecked exception from its computation method, or invoke `completeExceptionally(Throwable)`.

Specified by:

cancel in interface Future<V>

Parameters:

mayInterruptIfRunning - this value has no effect in the default implementation because interrupts are not used to control cancellation.

Returns:

true if this task is now cancelled

isDone

```
public final boolean isDone()
```

Description copied from interface: Future

Returns true if this task completed. Completion may be due to normal termination, an exception, or cancellation -- in all of these cases, this method will return true.

Specified by:

isDone in interface Future<V>

Returns:

true if this task completed

isCancelled

```
public final boolean isCancelled()
```

Description copied from interface: Future

Returns true if this task was cancelled before it completed normally.

Specified by:

isCancelled in interface Future<V>

Returns:

true if this task was cancelled before it completed

isCompletedAbnormally

```
public final boolean isCompletedAbnormally()
```

Returns true if this task threw an exception or was cancelled.

Returns:

true if this task threw an exception or was cancelled

isCompletedNormally

```
public final boolean isCompletedNormally()
```

Returns `true` if this task completed without throwing an exception and was not cancelled.

Returns:

`true` if this task completed without throwing an exception and was not cancelled

state

```
public Future.State state()
```

Description copied from interface: `Future`

Returns the computation state.

Specified by:

`state` in interface `Future<V>`

Returns:

the computation state

Since:

19

resultNow

```
public V resultNow()
```

Description copied from interface: `Future`

Returns the computed result, without waiting.

This method is for cases where the caller knows that the task has already completed successfully, for example when filtering a stream of Future objects for the successful tasks and using a mapping operation to obtain a stream of results.

```
    results = futures.stream()
                .filter(f -> f.state() == Future.State.SUCCESS)
                .map(Future::resultNow)
                .toList();
```

Specified by:

`resultNow` in interface `Future<V>`

Returns:

the computed result

Since:

19

exceptionNow

```
public Throwable exceptionNow()
```

Description copied from interface: `Future`

Returns the exception thrown by the task, without waiting.

This method is for cases where the caller knows that the task has already completed with an exception.

Specified by:

`exceptionNow` in interface `Future<V>`

Returns:

the exception thrown by the task

Since:

19

getException

```
public final Throwable getException()
```

Returns the exception thrown by the base computation, or a `CancellationException` if cancelled, or `null` if none or if the method has not yet completed.

Returns:

the exception, or `null` if none

completeExceptionally

```
public void completeExceptionally(Throwable ex)
```

Completes this task abnormally, and if not already aborted or cancelled, causes it to throw the given exception upon `join` and related operations. This method may be used to induce exceptions in asynchronous tasks, or to force completion of tasks that would not otherwise complete. Its use in other situations is discouraged. This method is overridable, but overridden versions must invoke `super` implementation to maintain guarantees.

Parameters:

ex - the exception to throw. If this exception is not a `RuntimeException` or `Error`, the actual exception thrown will be a `RuntimeException` with cause ex.

complete

```
public void complete(V value)
```

Completes this task, and if not already aborted or cancelled, returning the given value as the result of subsequent invocations of `join` and related operations. This method may be used to provide results for asynchronous tasks, or to provide alternative handling for tasks that would not otherwise complete normally. Its use in other situations is discouraged. This method is overridable, but overridden versions must invoke `super` implementation to maintain guarantees.

Parameters:

`value` - the result value for this task

quietlyComplete

```
public final void quietlyComplete()
```

Completes this task normally without setting a value. The most recent value established by `setRawResult(V)` (or `null` by default) will be returned as the result of subsequent invocations of `join` and related operations.

Since:

1.8

get

```
public final V get()
            throws InterruptedException,
                   ExecutionException
```

Waits if necessary for the computation to complete, and then retrieves its result.

Specified by:

`get` in interface `Future<V>`

Returns:

the computed result

Throws:

`CancellationException` - if the computation was cancelled

`ExecutionException` - if the computation threw an exception

`InterruptedException` - if the current thread is not a member of a ForkJoinPool and was interrupted while waiting

get

```
public final V get(long timeout,
                   TimeUnit unit)
            throws InterruptedException,
                   ExecutionException,
                   TimeoutException
```

Waits if necessary for at most the given time for the computation to complete, and then retrieves its result, if available.

Specified by:

`get` in interface `Future<V>`

Parameters:

`timeout` - the maximum time to wait

`unit` - the time unit of the timeout argument

Returns:

the computed result

Throws:

`CancellationException` - if the computation was cancelled

`ExecutionException` - if the computation threw an exception

`InterruptedException` - if the current thread is not a member of a ForkJoinPool and was interrupted while waiting

`TimeoutException` - if the wait timed out

quietlyJoin

```
public final void quietlyJoin()
```

Joins this task, without returning its result or throwing its exception. This method may be useful when processing collections of tasks when some have been cancelled or otherwise known to have aborted.

quietlyInvoke

```
public final void quietlyInvoke()
```

Commences performing this task and awaits its completion if necessary, without returning its result or throwing its exception.

quietlyJoin

```
public final boolean quietlyJoin(long timeout,
                                 TimeUnit unit)
                          throws InterruptedException
```

Tries to join this task, returning true if it completed (possibly exceptionally) before the given timeout elapsed and the current thread has not been interrupted.

Parameters:

`timeout` - the maximum time to wait

`unit` - the time unit of the timeout argument

Returns:

true if this task completed

Throws:

`InterruptedException` - if the current thread was interrupted while waiting

Since:

19

quietlyJoinUninterruptibly

```
public final boolean quietlyJoinUninterruptibly(long timeout,
                                                TimeUnit unit)
```

Tries to join this task, returning true if it completed (possibly exceptionally) before the given timeout elapsed.

Parameters:

`timeout` - the maximum time to wait

`unit` - the time unit of the timeout argument

Returns:

true if this task completed

Since:

19

helpQuiesce

```
public static void helpQuiesce()
```

Possibly executes tasks until the pool hosting the current task is quiescent. This method may be of use in designs in which many tasks are forked, but none are explicitly joined, instead executing them until all are processed.

reinitialize

```
public void reinitialize()
```

Resets the internal bookkeeping state of this task, allowing a subsequent `fork`. This method allows repeated reuse of this task, but only if reuse occurs when this task has either never been forked, or has been forked, then completed and all outstanding joins of this task have also completed. Effects under any other usage conditions are not guaranteed. This method may be useful when executing pre-constructed trees of subtasks in loops.

Upon completion of this method, `isDone()` reports `false`, and `getException()` reports `null`. However, the value returned by `getRawResult` is unaffected. To clear this value, you can invoke `setRawResult(null)`.

getPool

```
public static ForkJoinPool getPool()
```

Returns the pool hosting the current thread, or `null` if the current thread is executing outside of any ForkJoinPool.

This method returns `null` if and only if `inForkJoinPool()` returns `false`.

Returns:

the pool, or `null` if none

inForkJoinPool

```
public static boolean inForkJoinPool()
```

Returns `true` if the current thread is a `ForkJoinWorkerThread` executing as a ForkJoinPool computation.

Returns:

`true` if the current thread is a `ForkJoinWorkerThread` executing as a ForkJoinPool computation, or `false` otherwise

tryUnfork

```
public boolean tryUnfork()
```

Tries to unschedule this task for execution. This method will typically (but is not guaranteed to) succeed if this task is the most recently forked task by the current thread, and has not commenced executing in another thread. This method may be useful when arranging alternative local processing of tasks that could have been, but were not, stolen.

Returns:

`true` if unforked

getQueuedTaskCount

```
public static int getQueuedTaskCount()
```

Returns an estimate of the number of tasks that have been forked by the current worker thread but not yet executed. This value may be useful for heuristic decisions about whether to fork other tasks.

Returns:

the number of tasks

getSurplusQueuedTaskCount

```
public static int getSurplusQueuedTaskCount()
```

Returns an estimate of how many more locally queued tasks are held by the current worker thread than there are other worker threads that might steal them, or zero if this thread is not operating in a ForkJoinPool. This value may be useful for heuristic decisions about whether to fork other tasks. In many usages of ForkJoinTasks, at steady state, each worker should aim to maintain a small constant surplus (for example, 3) of tasks, and to process computations locally if this threshold is exceeded.

Returns:

the surplus number of tasks, which may be negative

getRawResult

```
public abstract V getRawResult()
```

Returns the result that would be returned by `join()`, even if this task completed abnormally, or `null` if this task is not known to have been completed. This method is designed to aid debugging, as well as to support extensions. Its use in any other context is discouraged.

Returns:

the result, or `null` if not completed

setRawResult

```
protected abstract void setRawResult(V value)
```

Forces the given value to be returned as a result. This method is designed to support extensions, and should not in general be called otherwise.

Parameters:

`value` - the value

exec

```
protected abstract boolean exec()
```

Immediately performs the base action of this task and returns true if, upon return from this method, this task is guaranteed to have completed. This method may return false otherwise, to indicate that this task is not necessarily complete (or is not known to be complete), for example in asynchronous actions that require explicit invocations of completion methods. This method may also throw an (unchecked) exception to indicate abnormal exit. This method is designed to support extensions, and should not in general be called otherwise.

Returns:

`true` if this task is known to have completed normally

peekNextLocalTask

```
protected static ForkJoinTask<?> peekNextLocalTask()
```

Returns, but does not unschedule or execute, a task queued by the current thread but not yet executed, if one is immediately available. There is no guarantee that this task will actually be polled or executed next. Conversely, this method may return null even if a task exists but cannot be accessed without contention with other threads. This method is designed primarily to support extensions, and is unlikely to be useful otherwise.

Returns:

the next task, or `null` if none are available

pollNextLocalTask

```
protected static ForkJoinTask<?> pollNextLocalTask()
```

Unschedules and returns, without executing, the next task queued by the current thread but not yet executed, if the current thread is operating in a ForkJoinPool. This method is designed primarily to support extensions, and is unlikely to be useful otherwise.

Returns:

the next task, or `null` if none are available

pollTask

```
protected static ForkJoinTask<?> pollTask()
```

If the current thread is operating in a ForkJoinPool, unschedules and returns, without executing, the next task queued by the current thread but not yet

executed, if one is available, or if not available, a task that was forked by some other thread, if available. Availability may be transient, so a `null` result does not necessarily imply quiescence of the pool this task is operating in. This method is designed primarily to support extensions, and is unlikely to be useful otherwise.

Returns:

a task, or `null` if none are available

pollSubmission

```
protected static ForkJoinTask<?> pollSubmission()
```

If the current thread is operating in a ForkJoinPool, unschedules and returns, without executing, a task externally submitted to the pool, if one is available. Availability may be transient, so a `null` result does not necessarily imply quiescence of the pool. This method is designed primarily to support extensions, and is unlikely to be useful otherwise.

Returns:

a task, or `null` if none are available

Since:

9

getForkJoinTaskTag

```
public final short getForkJoinTaskTag()
```

Returns the tag for this task.

Returns:

the tag for this task

Since:

1.8

setForkJoinTaskTag

```
public final short setForkJoinTaskTag(short newValue)
```

Atomically sets the tag value for this task and returns the old value.

Parameters:

`newValue` - the new tag value

Returns:

the previous value of the tag

Since:

1.8

compareAndSetForkJoinTaskTag

```
public final boolean compareAndSetForkJoinTaskTag(short expect,
                                                  short update)
```

Atomically conditionally sets the tag value for this task. Among other applications, tags can be used as visit markers in tasks operating on graphs, as in methods that check: `if (task.compareAndSetForkJoinTaskTag((short)0, (short)1))` before processing, otherwise exiting because the node has already been visited.

Parameters:

`expect` - the expected tag value

`update` - the new tag value

Returns:

`true` if successful; i.e., the current value was equal to `expect` and was changed to `update`.

Since:

1.8

adapt

```
public static ForkJoinTask<?> adapt(Runnable runnable)
```

Returns a new `ForkJoinTask` that performs the `run` method of the given `Runnable` as its action, and returns a null result upon `join()`.

Parameters:

`runnable` - the runnable action

Returns:

the task

adapt

```
public static <T> ForkJoinTask<T> adapt(Runnable runnable,
                                        T result)
```

Returns a new `ForkJoinTask` that performs the `run` method of the given `Runnable` as its action, and returns the given result upon `join()`.

Type Parameters:

`T` - the type of the result

Parameters:

`runnable` - the runnable action

`result` - the result upon completion

Returns:

the task

adapt

```
public static <T> ForkJoinTask<T> adapt
(Callable<? extends T> callable)
```

Returns a new `ForkJoinTask` that performs the `call` method of the given `Callable` as its action, and returns its result upon `join()`, translating any checked exceptions encountered into `RuntimeException`.

Type Parameters:

`T` - the type of the callable's result

Parameters:

`callable` - the callable action

Returns:

the task

adaptInterruptible

```
public static <T> ForkJoinTask<T> adaptInterruptible
(Callable<? extends T> callable)
```

Returns a new `ForkJoinTask` that performs the `call` method of the given `Callable` as its action, and returns its result upon `join()`, translating any checked exceptions encountered into `RuntimeException`. Additionally, invocations of `cancel` with `mayInterruptIfRunning true` will attempt to interrupt the thread performing the task.

Type Parameters:

`T` - the type of the callable's result

Parameters:

`callable` - the callable action

Returns:

the task

Since:

19

adaptInterruptible

```
public static <T> ForkJoinTask<T> adaptInterruptible
(Runnable runnable,
 T result)
```

Returns a new `ForkJoinTask` that performs the `run` method of the given `Runnable` as its action, and returns the given result upon `join()`, translating any checked exceptions encountered into `RuntimeException`. Additionally, invocations of `cancel` with `mayInterruptIfRunning true` will attempt to interrupt the thread performing the task.

Type Parameters:

`T` - the type of the result

Parameters:

`runnable` - the runnable action

`result` - the result upon completion

Returns:

the task

Since:

22

adaptInterruptible

```
public static ForkJoinTask<?> adaptInterruptible(Runnable runnable)
```

Returns a new `ForkJoinTask` that performs the `run` method of the given `Runnable` as its action, and returns null upon `join()`, translating any checked exceptions encountered into `RuntimeException`. Additionally, invocations of `cancel` with `mayInterruptIfRunning true` will attempt to interrupt the thread performing the task.

Parameters:

`runnable` - the runnable action

Returns:

the task

Since:

22

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other

countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.